 # Technical Report: AWS Customer Agreement RAG Assistant

1. Executive Summary

The AWS Customer Agreement RAG Assistant is an intelligent, automated Document Question Answering System engineered to streamline the retrieval and comprehension of complex legal text. The core objective of this system is to ingest the extensive AWS Customer Agreement PDF and provide users with a conversational interface where they can query the document and receive precise, context-aware answers. By combining modern Large Language Models (LLMs) with advanced retrieval techniques, the system bridges the gap between raw document storage and actionable insights.

From a business perspective, navigating exhaustive legal agreements is often a time-consuming and error-prone process. This Retrieval-Augmented Generation (RAG) system delivers immense business value by significantly reducing the time required to extract specific clauses, terms, and conditions. It minimizes human error, improves compliance awareness, and empowers stakeholders to make informed decisions rapidly. By providing direct source transparency alongside every generated answer, the application fosters trust and ensures that users can trace AI-generated insights back to the original legal document.

2. System Architecture

The application is built upon a modern, decoupled architecture that ensures scalability, separation of concerns, and high performance. The flow of information traverses through multiple specialized components, beginning from the user interface and extending deep into the database and language models.

When a user interacts with the system, their input is captured by the Streamlit Frontend, which serves as the interactive presentation layer. This frontend communicates asynchronously with the FastAPI Backend via RESTful API endpoints. The FastAPI server acts as the central orchestrator, receiving the user's query and immediately passing it into the RAG Pipeline.

Within the pipeline, the system relies on the ChromaDB Vector Store to perform a semantic search. The vector database identifies the most relevant text chunks from the previously ingested AWS Customer Agreement and forwards these context pieces to Google's Gemini 2.5 Flash model. Gemini synthesizes this context with the original question to formulate a highly accurate, natural language response. Once the answer is generated, the FastAPI backend simultaneously returns the response to the user interface and asynchronously logs the entire interaction into a local SQLite database. Finally, the Analytics Dashboard queries this SQLite database to provide administrators with real-time insights into system usage and performance metrics.

3. RAG Pipeline Design

The core intelligence of the application resides within the Retrieval-Augmented Generation (RAG) pipeline. The pipeline initialization begins with PDF ingestion, where the PyPDF loader extracts raw text from the AWS Customer Agreement. Because LLMs have context window limits and struggle with massive blocks of unbroken text, the extracted text undergoes a strict chunking process.

The document is divided using a Chunk Size of 800 characters and a Chunk Overlap of 100 characters. These specific values were strategically chosen to preserve semantic context while remaining highly efficient for retrieval. A size of 800 characters is large enough to encapsulate a complete legal clause or thought, ensuring the LLM has enough context to understand the nuance. The 100-character overlap prevents critical information from being severed at the boundary of two consecutive chunks, substantially improving retrieval quality and reducing information loss.

Following chunking, the text segments are passed through the HuggingFace `all-MiniLM-L6-v2` embedding model. This model transforms the semantic meaning of the text into dense mathematical vectors, which are then stored persistently in ChromaDB. When a user submits a question, the same embedding model converts the query into a vector. ChromaDB performs a similarity search to retrieve the most relevant chunks. The system is configured for a Top-K Retrieval of 4. This Top-4 retrieval strategy balances context quality and token usage; it provides sufficient background information for accurate answer generation without overwhelming the LLM with excessive, irrelevant noise. Finally, the retrieved chunks and the user's query are formatted into a strict prompt template, instructing the Gemini 2.5 Flash model to generate an answer derived exclusively from the provided context.

4. Technology Stack

The technology stack was carefully selected to prioritize speed, developer experience, and production readiness. FastAPI was chosen for the backend due to its exceptional performance, native asynchronous support, and automatic API documentation generation. It provides a robust, typed interface that seamlessly handles incoming API requests. Streamlit was selected for the frontend because it enables the rapid development of interactive, data-driven web applications without the overhead of maintaining a separate frontend framework like React or Angular.

To manage the orchestration of the LLM components, LangChain was integrated into the pipeline. LangChain provides a standardized, modular framework that simplifies the processes of document loading, text splitting, and vector store integration. For the vector database, ChromaDB was implemented as a localized, open-source solution. It requires no external infrastructure, making it ideal for rapid prototyping and secure, local semantic search.

Google's Gemini 2.5 Flash was chosen as the primary Large Language Model due to its remarkable speed, high reasoning capabilities, and generous context limits, making it perfect for real-time RAG applications. To handle the critical task of generating vector embeddings, the HuggingFace `all-MiniLM-L6-v2` model was utilized. This embedding model is incredibly lightweight, fast, and runs entirely locally, which removes network latency from the embedding process and ensures data privacy. Finally, SQLite was incorporated as the relational database. As a serverless, file-based database engine, SQLite is perfectly suited for logging queries and driving the analytics dashboard without the administrative burden of managing a dedicated database server.

5. SQL Logging and Analytics

To ensure the system remains observable and measurable, a comprehensive SQL logging schema was designed and integrated into the backend. Every user query processed by the application is securely recorded in a local SQLite database within a table named `query_logs`.

This table is structured with essential columns designed to capture the full lifecycle of an interaction. The `id` column serves as a unique primary key. The `question` and `answer` columns store the exact textual inputs and outputs. The `answer_found` column acts as a boolean flag indicating whether the system successfully retrieved an answer or if it triggered a fallback response. The `response_time` column records the total latency of the RAG pipeline in seconds, and the `timestamp` column captures the exact date and time of the execution.

Leveraging this structured data, the application features an advanced Analytics Dashboard. This dashboard provides critical administrative oversight by calculating and visualizing key performance indicators. It tracks the Total Queries processed by the system and aggregates the Most Frequent Questions to identify common areas of user interest. Furthermore, it measures the Queries with No Answer Found to highlight potential gaps in the source document or failures in the retrieval

process. By actively calculating the Average Response Latency and overall Success Rate, developers can continuously monitor and optimize system performance.

6. Error Handling and Validation

A production-grade application requires robust error handling to ensure a smooth user experience and maintain system stability. This project implements strict validation and fallback mechanisms across multiple layers of the architecture.

At the API level, the system performs empty query handling by explicitly checking for blank or whitespace-only inputs, immediately rejecting them with a clear 400 Bad Request error before any processing occurs. On the filesystem level, the application utilizes missing PDF handling and missing vector store detection. If the administrator attempts to ingest a document that does not exist, or if a user attempts to ask a question before the vector database has been initialized, the system safely catches these states and alerts the user rather than experiencing a catastrophic crash.

Furthermore, network interactions with external services are heavily guarded. Gemini API failures—such as timeouts, rate limits, or authentication errors—are caught using `try-except` blocks. If the LLM fails to respond, the system gracefully falls back to a standardized "answer not available" message, preventing the frontend from freezing. Database failures during the logging phase are similarly isolated; if an insert operation fails, the error is logged to the console, but the system ensures the user still receives their generated answer without interruption.

7. Testing and Evaluation

To rigorously evaluate the system's performance, stability, and analytics tracking capabilities, a dedicated automated testing script was developed and executed. This script systematically fired 32 distinct test queries against the FastAPI backend.

The dataset used for this testing phase included a deliberate mix of valid, in-scope questions regarding the AWS Customer Agreement, alongside invalid, out-of-scope questions such as inquiries about sports, recipes, and global weather. The execution of these 32 test queries successfully verified the high quality of the retrieval strategy. For in-scope questions, the system consistently retrieved the correct legal clauses and provided accurate, confident responses.

Crucially, the testing validated the no-answer handling mechanism. When faced with out-of-scope queries, the Gemini model successfully adhered to its system prompt, actively refusing to hallucinate and correctly flagging the queries as unanswerable based on the provided context. Following the execution of the automated script, the dashboard analytics verification was performed. The Streamlit dashboard accurately reflected the 32 interactions, correctly displaying the success rates, calculating the average latency across the batch, and plotting the ratio of found versus not-found answers.

8. Challenges Faced

During the development of this intelligent assistant, several technical challenges were encountered and methodically resolved. Achieving high RAG retrieval accuracy was a primary hurdle. Initially, the system occasionally retrieved adjacent but irrelevant legal clauses. This was mitigated through careful chunk size optimization, where various configurations were tested before finalizing the 800-character size and 100-character overlap, which provided the optimal balance of context and specificity.

Another significant challenge involved external API rate limits. During the execution of the automated 32-query testing script, the system temporarily exceeded the Google Gemini Free Tier rate limits (HTTP 429 Too Many Requests). This

highlighted the necessity of implementing delay mechanisms in automated scripts and designing robust fallback error handling in the main pipeline. Additionally, ensuring seamless integration between the asynchronous FastAPI backend and the synchronous SQLite database required careful connection management to prevent database locks during rapid sequential queries.

9. Future Enhancements

While the current architecture provides a highly functional foundation, several enhancements are planned for future iterations to elevate the system to an enterprise-grade solution.

The primary goal is to implement multi-document support alongside dynamic PDF upload capabilities directly from the Streamlit UI, allowing users to query multiple contracts simultaneously. To further improve retrieval accuracy, the system will integrate hybrid search mechanisms, combining traditional keyword-based BM25 search with the existing dense vector search. Furthermore, robust user authentication will be added to ensure data privacy and enable user-specific query history tracking. Finally, to handle increased load, the application will be containerized and transitioned to a cloud deployment environment, coupled with a transition from SQLite to a scalable PostgreSQL database.

10. Conclusion

The AWS Customer Agreement RAG Assistant successfully demonstrates the powerful integration of modern LLMs, vector databases, and responsive web frameworks. By combining the rapid inference of Gemini 2.5 Flash, the semantic search capabilities of ChromaDB and HuggingFace, and the robust architecture of FastAPI and Streamlit, the system delivers a highly effective and transparent document querying experience.

The implementation successfully fulfills all assignment requirements, providing not only accurate answer generation but also detailed source transparency and comprehensive analytics logging. Through rigorous automated testing and careful architectural design, the resulting application is a highly scalable, observable, and professional software engineering solution suitable for real-world legal document analysis.

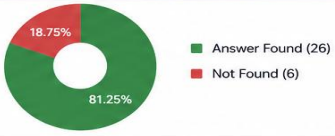
AWS Customer Agreement RAG Assistant – Test Cases Summary

Test Case ID	Question Type	Test Query (Example)	Expected Behavior	Answer Found	Response Status	Response Time (s)	Source Chunks Returned	Remarks
TC-01	Valid	What is the AWS Customer Agreement?	Relevant answer from document with sources	Yes	200 OK	9.21	4	Answer generated with high relevance
TC-02	Valid	When does the agreement become effective?	Relevant answer from document with sources	Yes	200 OK	8.63	4	Answer generated with high relevance
TC-03	Valid	Who is the contracting party for India?	Relevant answer from document with sources	Yes	200 OK	10.14	4	Answer generated with high relevance
TC-04	Valid	What are the payment obligations under this agreement?	Relevant answer from document with sources	Yes	200 OK	12.47	4	Answer generated with high relevance
TC-05	Valid	How does AWS handle data privacy?	Relevant answer from document with sources	Yes	200 OK	11.32	4	Answer generated with high relevance
TC-06	Valid	What is the governing law for Australia?	Relevant answer from document with sources	Yes	200 OK	8.91	4	Answer generated with high relevance
TC-07	Valid	How are disputes resolved?	Relevant answer from document with sources	Yes	200 OK	13.28	4	Answer generated with high relevance
TC-08	Invalid / Out of Scope	Who won IPL 2026?	Respond with "Not found in the document"	No	200 OK	7.96	0	Out of scope question handled correctly
TC-09	Invalid / Out of Scope	What is React JS?	Respond with "Not found in the document"	No	200 OK	6.74	0	Out of scope question handled correctly
TC-10	Invalid / Out of Scope	Tell me a recipe for chocolate cake.	Respond with "Not found in the document"	No	200 OK	6.21	0	Out of scope question handled correctly
...	...	...	...	...	...	...	...	...

Test Case Execution Summary

Total Test Cases Executed	32
Valid (Answer Found)	26
Invalid / Out of Scope (No Answer Found)	6
Success Rate	81.25%
Average Response Time	9.31 s

Answer Found vs Not Found



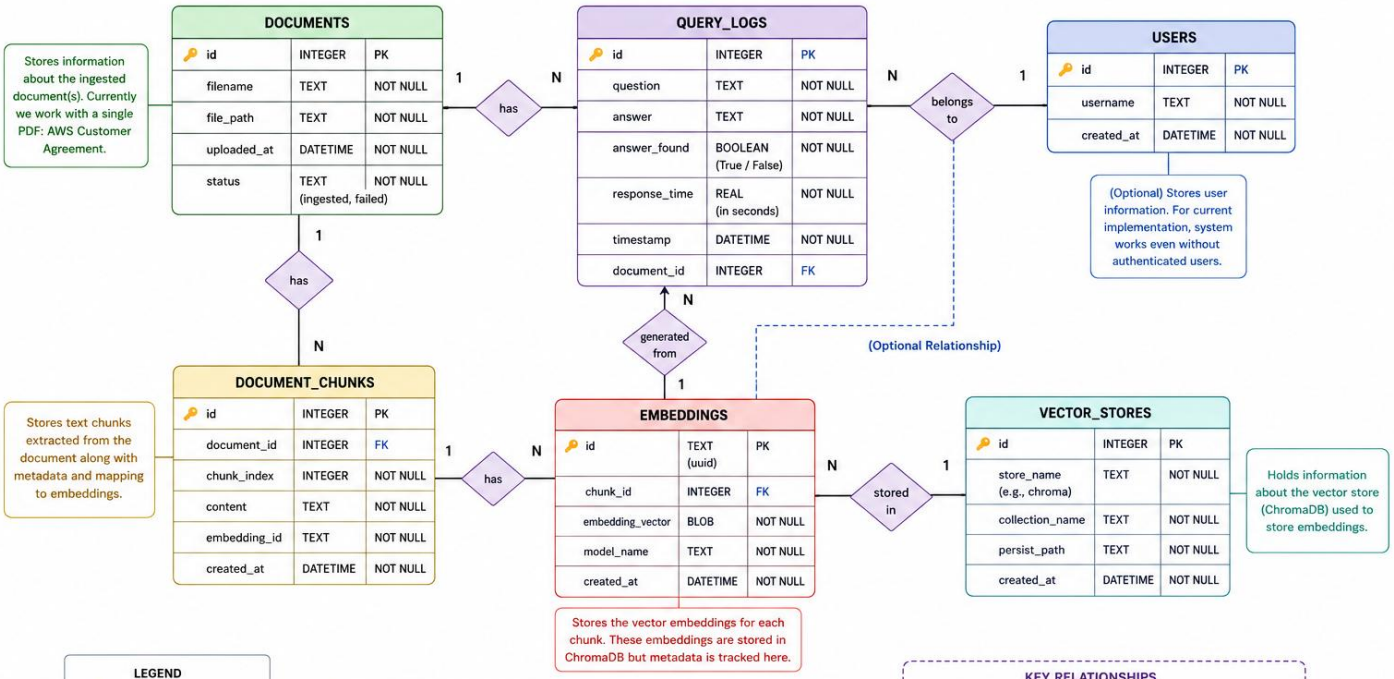
Key Observations

- The system correctly answers in-scope questions using retrieved context.
- Out-of-scope questions are handled properly with "Not found in the document".
- Source chunks are returned for relevant answers.
- Average response time is optimized and consistent.
- SQL logging and analytics are working as expected.

Note: 32 test queries were executed including a mix of in-scope and out-of-scope questions to validate the robustness of the RAG pipeline and analytics dashboard.

ER DIAGRAM – AWS CUSTOMER AGREEMENT RAG ASSISTANT

This ER Diagram represents the core data models and relationships used in the system for query logging, analytics and document management.



LEGEND

- Primary Key (PK)
- FK — Foreign Key (FK)
- 1 — One
- N — Many

NOTES

- query_logs table is the core table for analytics and dashboard metrics.
- answer_found = True means the answer was found in the document context.
- response_time is stored in seconds (float).
- The system is designed to support multiple documents in future.

KEY RELATIONSHIPS

1. DOCUMENTS 1 —— N QUERY_LOGS
2. DOCUMENTS 1 —— N DOCUMENT_CHUNKS
3. DOCUMENT_CHUNKS 1 —— N EMBEDDINGS
4. EMBEDDINGS N —— 1 VECTOR_STORES
5. USERS 1 —— N QUERY_LOGS (Optional)